DAY 1 – FOUNDATION

Goal

Understand how **iLO firmware = Embedded Linux system running on server BMC**

CONCEPTS (DETAILED)

1. Embedded Linux in Servers

- What is BMC (Baseboard Management Controller)
- Role of iLO in HPE servers
- Firmware vs OS vs Hypervisor

Architecture:
Server Hardware → BMC → Embedded Linux → iLO Services

2. GNU Toolchain (Cross Compilation)

- Native vs Cross compilation
- Toolchain components:
    - gcc
    - binutils
    - libc
- Target architectures:
    - ARM (common in BMC like iLO)

3. Development Environment Setup

- QEMU vs Raspberry Pi
- Why QEMU for enterprise training
- SSH-based workflow

HANDS-ON (STEP-BY-STEP)

LAB 1: Install Toolchain

sudo apt install gcc-arm-linux-gnueabihf

LAB 2: Cross Compile Program

```
#include <stdio.h>
int main() {
   printf("Hello from iLO-like firmware\n");
   return 0;
}
```

arm-linux-gnueabihf-gcc hello.c -o hello_arm

LAB 3: Run via QEMU

qemu-arm -L /usr/arm-linux-gnueabihf/ ./hello_arm

LAB 4: SSH into Target

- Setup minimal rootfs (pre-provided)
- SSH into QEMU system

iLO MAPPING

Training Concept	iLO Reality
Cross compile	Firmware build for BMC chip
ARM binary	iLO runs on ARM processor
SSH access	Remote management interface
Embedded Linux	iLO firmware OS

DAY 1 OUTCOME

✔ Understand firmware architecture
✔ Able to compile for target hardware
✔ Access embedded system remotely

DAY 2 – SYSTEM BUILD (OS CREATION)

Goal

Build **your own iLO-like OS (Kernel + RootFS)**

CONCEPTS

1. Linux Kernel Deep Dive

- Kernel layers:
    - Process mgmt
    - Memory mgmt
    - Device drivers
- Kernel config (`menuconfig`)

2. Root Filesystem

Structure:
/bin /sbin /etc /proc /sys /dev

- BusyBox (tiny Linux tools)

3. QEMU System Emulation

- Full system emulation (ARM board)
- Booting kernel manually

HANDS-ON

LAB 1: Build Kernel

make ARCH=arm menuconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-

LAB 2: Build BusyBox RootFS

make menuconfig
make install

LAB 3: Boot System in QEMU

qemu-system-arm -kernel zImage -initrd rootfs.cpio

iLO MAPPING

Training	iLO
Kernel build	Firmware core
RootFS	iLO OS environment
BusyBox	Lightweight utilities
Boot in QEMU	Server boot simulation

DAY 2 OUTCOME

✔ Built full OS from scratch
✔ Understand boot lifecycle
✔ Able to debug boot failures

DAY 3 – BOOT + SERVICES

Goal

Control **how iLO boots + runs services**

CONCEPTS

1. Boot Flow

ROM → Bootloader → Kernel → Init → Services

2. U-Boot Bootloader

- Boot scripts
- Environment variables
- Kernel loading

3. systemd (Enterprise Level)

- Units (service, target)
- Boot dependency graph
- Logging via journalctl

HANDS-ON

LAB 1: U-Boot Build

make CROSS_COMPILE=arm-linux-gnueabihf-

LAB 2: Boot Script

setenv bootargs console=ttyS0
bootz ${kernel_addr}

LAB 3: Create systemd Service

[Service]
ExecStart=/usr/bin/ilo-service

LAB 4: Logging

journalctl -u ilo-service

iLO MAPPING

Training	iLO
U-Boot	Server firmware bootloader
Boot args	Hardware config
systemd	iLO services manager
Logging	Remote diagnostics

MINI PROJECT (DAY 3)

Build “Mini iLO Service”

- Runs on boot
- Exposes system info
- Logs status

DAY 3 OUTCOME

✔ Control boot process
✔ Build service like iLO feature
✔ Debug system behavior

DAY 4 – YOCTO + PRODUCTION

Goal

Build **production-grade firmware like HPE pipelines**

CONCEPTS

1. Yocto Architecture

- Layers (meta, BSP)
- Recipes (.bb files)
- BitBake engine

2. Image Customization

- Add packages
- Modify rootfs
- Kernel customization

3. Debugging & Optimization

- BitBake logs
- Dependency issues
- Build performance

HANDS-ON

LAB 1: Build Yocto Image

bitbake core-image-minimal

LAB 2: Add SSH

IMAGE_INSTALL += "openssh"

LAB 3: Create Custom Recipe

DESCRIPTION = "iLO Service"

LAB 4: Kernel Customization

- Modify config via `.bbappend`

iLO MAPPING

Training	iLO Reality
Yocto build	Firmware build system
Recipes	Feature modules
Layers	Product customization
BitBake	Build pipeline
Image	iLO firmware image

FINAL CAPSTONE PROJECT

“Build Mini iLO Firmware”

Features:

- Custom Linux image
- Boot via U-Boot
- systemd service
- SSH enabled
- Custom monitoring app

Debugging + Failure Basic's

Kernel panic debugging

Boot failure debugging

systemd service failure

DAY 4 OUTCOME

✔ Build production firmware
✔ Customize OS like HPE
✔ Understand enterprise pipelines